WELT 1 · DOPPELSTUNDEN 8–10 · LEVEL 1.1 BIS 1.3

W1·1 – Die Heldenschmiede: Klassen und Objekte

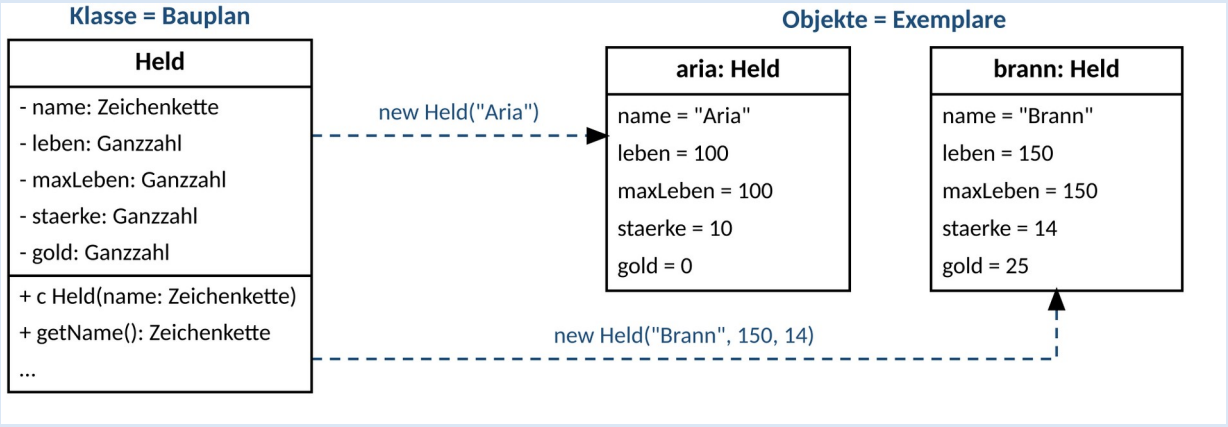
Datei: `AQ_W1_L1_Heldenschmiede_Geruest.txt` · IDE-Level 1.1 bis 1.3: Basis · Erweiterung · Experte · Papier-Compiler: Aufgabe 8 · Im Lab: **Level 1.1 bis 1.3**
Niveau (KC 2017): Grund- und Leistungskurs – Aufgabe 7 (Klassendiagramm mit Beziehungen) nur Leistungskurs (eA)

Die Akademie liegt hinter euch. Bevor euer Studio das erste Dungeon baut, braucht es Figuren: Helden mit Leben und Stärke, Monster zum Bekämpfen und Schätze zum Einsammeln. In der **Heldenschmiede** entsteht jede Figur nach einem Bauplan. In Java heißt so ein Bauplan **Klasse**.

Im Abitur musst du Klassen lesen, erläutern, als Klassendiagramm darstellen und von Hand implementieren. Darum wechseln sich auf diesem Blatt IDE-Level und Papier-Aufgaben ab.

Infoteil 1: Klasse und Objekt – Bauplan und Exemplar

- Eine **Klasse** ist ein Bauplan. Sie legt fest, welche Eigenschaften (**Attribute**) und Fähigkeiten (**Methoden**) alle Helden haben.
- Ein **Objekt** ist ein konkretes Exemplar, das nach diesem Bauplan mit `new` erzeugt wird. Jedes Objekt hat **eigene Attributwerte** – seinen Zustand.
- Aus einer Klasse kannst du beliebig viele Objekte erzeugen. Schaden für Aria ändert nichts an Brann.



Infoteil 2: Bestandteile einer Klasse – am Beispiel Held

Hier siehst du den Anfang der Klasse `Held`. Die Buchstaben in den Kommentaren gehören zur Tabelle darunter.

```
public class Held {

    private String name;                  // (A) Attribute
    private int leben;
    private int maxLeben;
    private int staerke;
    private int gold;

    public Held(String name) {            // (B) Konstruktor
        this.name = name;                 // (E) this
        maxLeben = 100;
        leben = 100;
        staerke = 10;
        gold = 0;
    }

    public String getName() {             // (C) Getter
        return name;
    }

    public void nimmSchaden(int schaden) { // (D) veraendernde Methode
        // ... das programmierst du in Level 1.1
    }
}
```

	Begriff	Bedeutung
(A)	**Attribut**	Eigenschaft, die jedes Objekt hat – mit eigenem Wert pro Objekt. `private` (–): nur innerhalb der Klasse sichtbar.
(B)	**Konstruktor**	Wird bei `new` aufgerufen und setzt die Startwerte. Er heißt wie die Klasse und hat **keinen** Rückgabetyp.
(C)	**Getter**	Sondierende Methode: liefert einen Attributwert, ändert nichts. Ein **Setter** (z. B. `setGold(int betrag)`) würde ein Attribut direkt überschreiben.
(D)	**verändernde Methode**	Ändert den Zustand des Objekts – kontrolliert, z. B. „leben nie unter 0“.
(E)	**this**	Verweist auf das eigene Objekt. `this.name` ist das Attribut, `name` allein der Parameter.
	Sichtbarkeit	`private` (–) für Attribute, `public` (+) für Methoden und Konstruktoren.

Merke: Held hat Getter, aber bewusst **keine Setter** für `leben` und `gold`. Von außen geht es nur über `nimmSchaden`, `heilen` und `erhalteGold`. So kann das Objekt selbst dafür sorgen, dass sein Zustand gültig bleibt. Das nennt man **Kapselung**.

Infoteil 3: Klassenkarte und Objektkarte (EH § 6)

Held
- name: Zeichenkette - leben: Ganzzahl - maxLeben: Ganzzahl - staerke: Ganzzahl - gold: Ganzzahl - inventar: Reihung von Gegenstand - anzahl: Ganzzahl - waffe: Waffe
+ c Held(name: Zeichenkette) + c Held(name: Zeichenkette, maxLeben: Ganzzahl, staerke: Ganzzahl) + getName(): Zeichenkette + getLeben(): Ganzzahl + getMaxLeben(): Ganzzahl + getStaerke(): Ganzzahl + getGold(): Ganzzahl + getAnzahl(): Ganzzahl + getWaffe(): Waffe + erhalteGold(betrag: Ganzzahl): nichts + istBesiegt(): Wahrheitswert + nimmSchaden(schaden: Ganzzahl): nichts + heilen(punkte: Ganzzahl): nichts + angriffswert(): Ganzzahl + angreifen(m: Monster): nichts + aufnehmen(g: Gegenstand): Wahrheitswert + gesamtWert(): Ganzzahl + wertvollsterGegenstand(): Gegenstand + enthaelt(name: Zeichenkette): Wahrheitswert + ausruesten(w: Waffe): nichts + trinken(t: Trank): nichts + inventarAusgeben(): nichts

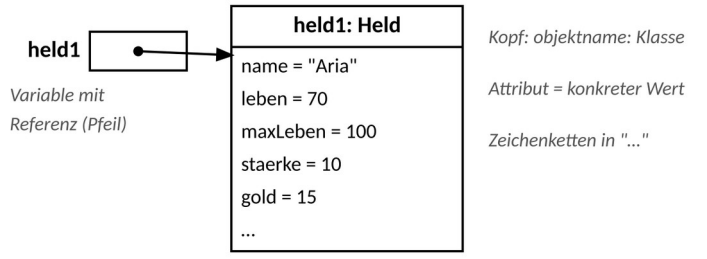
Abbildung 1: Klassenkarte Held (vollständig)

Klassenkarte – so im Abitur:

- Oben der Klassenname, darunter die Attribute, unten Konstruktoren und Methoden.
- Sichtbarkeit ist Pflicht: **–** privat, **+** öffentlich.
- Attribute: `- leben: Ganzzahl`
- Konstruktoren mit vorangestelltem **c**: `+ c Held(name: Zeichenkette)`
- Methoden mit Signatur: `+ getLeben(): Ganzzahl`, ohne Rückgabe: `: nichts`
- Nicht gezeigte Elemente werden durch **…** ersetzt.
- Typnamen deutsch: Ganzzahl, Zeichenkette, Wahrheitswert, Reihung von …

Objektkarte – zeigt ein Objekt zu einem bestimmten Zeitpunkt:

- Kopf: `objektname: Klasse`, z. B. held1: Held
- Jedes Attribut mit konkretem Wert: `leben = 70`, Zeichenketten in Anführungszeichen.
- Eine Variable, die auf ein Objekt verweist, zeichnest du als Kästchen mit Pfeil (Referenz). Kein Objekt: **null**.



Aufgabe 0: Heldenkarten aus der Schmiede

PAPIER ★

Bevor ihr programmiert, wird geschmiedet – unplugged, ohne Gerät.

a) Jede Person erfindet einen Helden und füllt eine Objektkarte aus. Denk dir einen Objektnamen aus (z. B. held1).

_____ : Held
name = _____
leben = _____
maxLeben = _____
staerke = _____
gold = _____

_____ : Held
name = _____
leben = _____
maxLeben = _____
staerke = _____
gold = _____

b) Legt eure Karten im Team nebeneinander. Was haben alle Karten gemeinsam, was unterscheidet sie? Ordne die Begriffe **Klasse**, **Objekt**, **Attribut** und **Attributwert** zu.

c) Die Lehrkraft ruft Aktionen auf, z. B. „Alle Helden mit Stärke über 12: nimmSchaden(30)“ oder „heilen(50)“. Aktualisiere deine Objektkarte. Achte auf die Regeln: leben nie unter 0, nie über maxLeben.

d) Entwerft gemeinsam die Klassenkarte für **Monster**: Ein Monster hat einen Namen, Leben und Stärke. Es kann Schaden nehmen, besiegt sein und einen Helden angreifen. Vergleicht danach mit der Klassenkarte im Gerüst.

Aufgabe 1: Die Schmiede öffnet – Level 1.1

IDE ★ – ★★★

a) Lege einen Workspace **AlgoQuest_W1** an und darin eine Datei mit dem Inhalt von `AQ_W1_L1_Heldenschmiede_Geruest.txt`. Lies die ANLEITUNG oben in der Datei.

b) Lege in der Klasse `Held` die Attribute **name, leben, maxLeben, staerke** und **gold** an – alle `private`. Die Attribute schreibst du selbst, die Klassenkarte (Abbildung 1) zeigt dir die Typen.

c) Implementiere den Konstruktor `Held(String name)` und die Getter `getName`, `getLeben`, `getMaxLeben`, `getStaerke` und `getGold`.

d) Implementiere `istBesiegt()`, `nimmSchaden(int schaden)` (leben nie unter 0), `heilen(int punkte)` (nie über maxLeben) und `erhalteGold(int betrag)` (nur, wenn betrag > 0).

e) Starte im Testrunner die Testklasse `L11HeldTest`. Die Stufe erkennst du am Anfang des Testnamens:

Stufe	Tests in L11HeldTest beginnen mit …
★ Basis	`basis_`
★★ Erweiterung	`erweiterung_`
★★★ Experte	`experte_`

Tipp: Denk wie ein Tester: Was passiert bei 0? Bei negativen Werten? Genau an der Grenze, z. B. bei leben = 1 und schaden = 1?

Aufgabe 2: Klasse erläutern – wie im Abitur

PAPIER ★★

Im Abitur beginnt fast jede Aufgabe mit einer Klassenkarte, die du erläutern sollst. Abbildung 2 zeigt die Klasse Monster.

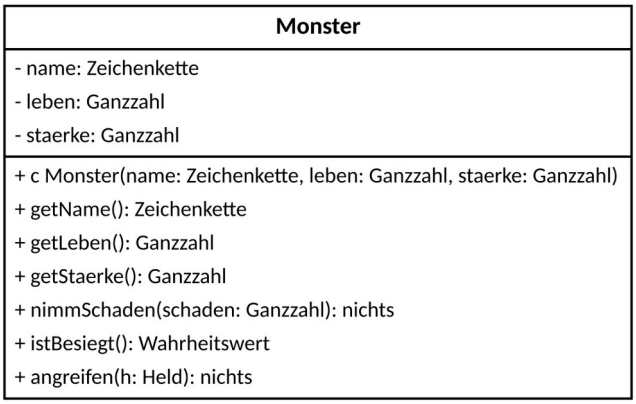


Abbildung 2: Klassenkarte Monster

a) Erläutere den grundsätzlichen Aufbau der Klasse Monster. Gehe dabei auf die Attribute, den Konstruktor, die Methoden und die Bedeutung der Zeichen + und − ein. **[4 BE]**

b) Begründe, warum das Attribut leben als privat deklariert ist, obwohl sich sein Wert im Spiel ständig ändert. **[2 BE]**

c) Gib an, welche Methoden sondierend und welche verändernd sind. **[2 BE]**

Achtung Abi: Operator „erläutern“ (AFB II)

Einen Sachverhalt nachvollziehbar und verständlich machen – mit Begründungen und Beispielen. Eine reine Aufzählung der Attribute reicht **nicht**. Zeitbudget: ca. 2,5 Minuten pro BE, hier also etwa 20 Minuten.

Infoteil 4: Überladene Konstruktoren und this

Eine Klasse darf **mehrere Konstruktoren** haben, wenn sich ihre Parameterlisten unterscheiden (Anzahl oder Typen). Beim `new` wählt Java den passenden Konstruktor aus. Beispiel (gehört nicht zum Projekt):

```
public class Tuer {
    private int nummer;
    private boolean offen;

    public Tuer(int nummer) {
        this.nummer = nummer;      // Attribut = Parameter
        this.offen = false;
    }

    public Tuer(int nummer, boolean offen) {
        this.nummer = nummer;
        this.offen = offen;
    }
}
```

`new Tuer(7)` ruft den ersten Konstruktor auf, `new Tuer(3, true)` den zweiten. In der Klassenkarte stehen beide mit **c**: `+ c Tuer(nummer: Ganzzahl)` und `+ c Tuer(nummer: Ganzzahl, offen: Wahrheitswert)`.

Typischer Fehler: Ohne `this` weist `nummer = nummer;` den Parameter sich selbst zu. Das Attribut behält seinen Standardwert 0 – und kein Compiler warnt dich.

Aufgabe 3: Zwei Wege in die Schmiede – Level 1.2, Teil 1

IDE ★

a) Implementiere den zweiten Konstruktor `Held(String name, int maxLeben, int staerke)`. Der Held startet mit vollem Leben (leben = maxLeben), alle anderen Startwerte wie beim ersten Konstruktor. Die Parameter heißen wie die Attribute – denk an `this`!

b) Prüfe deine Methode `heilen`: Funktioniert sie auch für einen Helden mit maxLeben 150? Welcher Test in `L12MonsterTest` prüft genau das?

c) Gib ohne Gerät an, welcher Konstruktor jeweils aufgerufen wird – oder ob der Compiler meckert.

Aufruf	Welcher Konstruktor? Oder Fehler – warum?
`new Held("Aria")`	
`new Held("Brann", 150, 14)`	
`new Held("Cora", 80)`	

Aufgabe 4: Der Doppelgänger

PAPIER ★★

Im Dungeon taucht ein Doppelgänger auf. Ist es dieselbe Heldin – oder nur eine, die genauso aussieht?

```
1  Held a = new Held("Aria");
2  Held b = a;
3  b.nimmSchaden(30);
4  Held c = new Held("Aria");
5  Held d = null;
6  println(a.getLeben());
7  println(c.getLeben());
8  println(a == b);
9  println(a == c);
```

a) Stelle den Zustand nach Zeile 5 dar: alle Variablen als Kästchen mit Referenzpfeil und alle Objekte als Objektkarten. Gib an, wie viele Held-Objekte es gibt. **[3 BE]**

b) Gib die vier Ausgaben an und begründe jede kurz. **[3 BE]**

c) Erläutere, was passiert, wenn als nächste Zeile `d.nimmSchaden(10);` ausgeführt wird. **[2 BE]**

Aufgabe 5: Monster und Kampf – Level 1.2, Teil 2

IDE ★ – ★★★

a) Vervollständige die Klasse `Monster`: Lege die Attribute selbst an (Abbildung 2), dann Konstruktor und Getter.

b) Implementiere `nimmSchaden` (leben nie unter 0), `istBesiegt` und `angreifen(Held h)`: Der Held nimmt Schaden in Höhe der Stärke des Monsters.

c) In `Held`: Implementiere `angriffswert()` – vorerst ist das einfach die Stärke – und `angreifen(Monster m)`.

d) **Vorhersage ohne Gerät:** Aria (100 Leben, Stärke 10) kämpft gegen einen Goblin (30 Leben, Stärke 6). Aria schlägt zuerst, ein noch lebender Goblin schlägt immer zurück. Nach wie vielen Runden ist der Goblin besiegt, wie viel Leben hat Aria dann? Prüfe deine Vorhersage mit dem Test `experte_kampfBisZumSieg`.

---

e) Starte `L12MonsterTest` (Stufen wie in Aufgabe 1) und danach das Hauptprogramm: Aria gegen den Goblin – diesmal mit Würfel.

Denkanstoß: `angreifen` ruft `m.nimmSchaden(…)` auf. Warum schreibst du nicht einfach `m.leben = m.leben - angriffswert();`?

Infoteil 5: Assoziation und Reihung von Objekten

- Ein Held besitzt Gegenstände: Er hat das Attribut `inventar` vom Typ **Reihung von Gegenstand**. Eine solche Beziehung zwischen zwei Klassen heißt **Assoziation**.
- Im Klassendiagramm (EH § 6) zeichnest du sie als **einfache Linie** zwischen den Klassenkarten – ohne Beschriftung, ohne Kardinalitäten. Wir zeichnen sie, wenn eine Klasse ein Attribut vom Typ der anderen Klasse hat (auch als Reihung).
- `new Gegenstand[5]` erzeugt fünf **leere Plätze** (null) – noch keinen einzigen Gegenstand!
- Das Attribut `anzahl` merkt sich, wie viele Plätze belegt sind. Belegt sind die Indizes 0 bis anzahl – 1, ein neuer Gegenstand kommt an den Index anzahl.

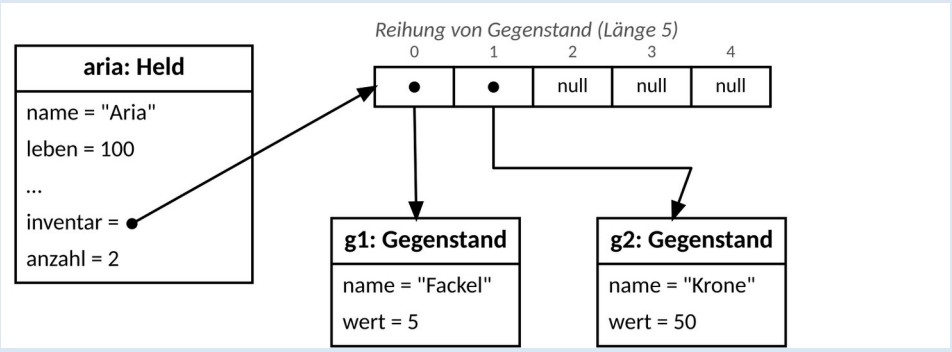


Abbildung 3: aria nach zwei Aufrufen von aufnehmen

Achtung Abi: Leer-, Voll- und null-Abfragen

- Vor dem Einfügen prüfen: Ist das Inventar voll (`anzahl == inventar.length`)? Ist der Parameter `null`?
- Schleifen über das Inventar laufen bis **anzahl − 1**, nicht bis inventar.length − 1 – sonst greifst du auf `null` zu.
- Namen mit `equals` vergleichen, nie mit `==`.

Aufgabe 6: Schätze und Inventar – Level 1.3

IDE ★ – ★★★

a) Vervollständige die Klasse `Gegenstand`: Konstruktor, Getter und `beschreibung()`, z. B. "Fackel (5 Gold)".

b) Lege in `Held` die Attribute `inventar` und `anzahl` selbst an und initialisiere sie in **beiden** Konstruktoren (Länge 5).

c) Implementiere `getAnzahl`, `aufnehmen` (false bei null oder vollem Inventar), `gesamtWert` und `enthaelt` (mit equals!).

d) **Stopp:** `wertvollsterGegenstand()` kommt in Aufgabe 8 – erst auf Papier! Lass die Methode bis dahin unverändert.

e) Starte `L13InventarTest`. Basis und Erweiterung schaffst du jetzt, die Stufe Experte erst nach Aufgabe 8.

Aufgabe 7: Die Karte der Schmiede ◆ nur LK (eA)

PAPIER ★★

a) Zeichne ein Klassendiagramm mit den Klassen Held, Monster und Gegenstand nach EH § 6. Bei Held genügen alle Attribute bis Level 1.3, die Konstruktoren und mindestens vier Methoden – den Rest ersetzt du durch „…“. Monster und Gegenstand zeichnest du vollständig. **[5 BE]**



b) Begründe, warum zwischen Held und Gegenstand eine Linie gezeichnet wird, zwischen Held und Monster aber nicht – obwohl Held die Methode angreifen(m: Monster): nichts hat. **[2 BE]**

Aufgabe 8: Der Papier-Compiler

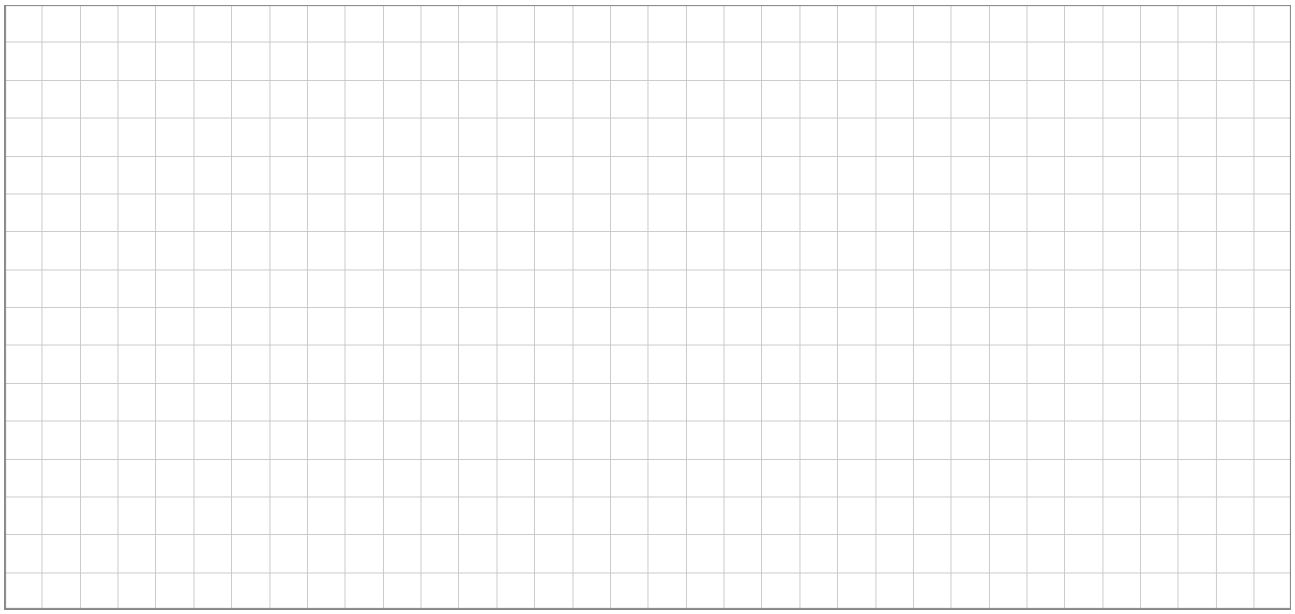
PAPIER → IDE ★★★

Regeln des Papier-Compilers

1. Schreibe die Methode **vollständig von Hand** – ohne Gerät, ohne Spickzettel außer den Werkzeugkarten.
2. Gehe den Abi-Check durch (unten).
3. Tippe deine Lösung **unverändert** ab – Zeichen für Zeichen, auch mit Fehlern!
4. Starte `L13InventarTest`. Alle Tests grün beim ersten Lauf: **Papier-Compiler bestanden**. Sonst korrigierst du, bis alles grün ist.
5. Jeder Fehler kommt ins Fehlerlogbuch – erst dann korrigierst du in der IDE.

Auftrag: Implementiere die Methode `wertvollsterGegenstand(): Gegenstand` der Klasse Held. Sie liefert den Gegenstand mit dem höchsten Wert. Ist das Inventar leer, liefert sie null. Haben mehrere Gegenstände den höchsten Wert, liefert sie den ersten davon.

a) Implementiere die Methode in Java. **[5 BE]**



b) Abi-Check – hake ab, bevor du abtippst:

- ☐ Signatur wie gefordert, Rückgabe in **jedem** Pfad
- ☐ Leer-Abfrage vor dem ersten Zugriff auf das Inventar
- ☐ Schleifengrenzen: nur belegte Plätze (bis anzahl − 1)
- ☐ Gleichstand: Liefert die Methode wirklich den **ersten**?
- ☐ Das Inventar ist am Ende unverändert

c) Ergebnis nach dem Abtippen: ☐ alle Tests grün beim ersten Lauf ☐ grün nach Korrektur

Fehlerlogbuch

Nr.	Was war falsch?	Art: Syntax / Logik / Abi-Check	So vermeide ich es
1			
2			
3			

🎯 Im Lab üben: Level 1.1 bis 1.3 – Punkte gibt es dort automatisch

Im **Algo Quest Lab** (algoquest.frankjuchim.de) findest du auf der Weltkarte von Welt 1 drei Level zu diesem Blatt. Die Aufgaben werden automatisch geprüft, Punkte gibt es beim ersten Lösen.

- **Level 1.1 „Die Schmiede öffnet – Klasse und Objekt“** – zu Infoteil 1–3 und Aufgaben 0–2
- **Level 1.2 „Doppelgänger – Konstruktoren und Referenzen“** – zu Infoteil 4 und Aufgaben 3–5
- **Level 1.3 „Der Rucksack – Reihung von Objekten“** – zu Infoteil 5 und Aufgaben 6–8

Die IDE-Level 1.1 bis 1.3 programmierst du weiterhin in der Online-IDE.

Für Schnelle: Die Arena

Baue eine Kampfsimulation mit der Klasse `Wuerfel` aus dem Gerüst. Ändere dafür **nicht** die Klassen Held und Monster.

- Schreibe eine Klasse `Arena` mit der Methode `public static boolean kampf(Held h, Monster m)`. In jeder Runde würfelt der Held mit `Wuerfel.wuerfeln(6)`: Bei einer 6 greift er zweimal an (kritischer Treffer), bei einer 1 verfehlt er, sonst greift er normal an. Danach greift das Monster an, falls es noch nicht besiegt ist. Rückgabe: true, wenn das Monster besiegt ist. Baue eine Notbremse nach 100 Runden ein.
- Lass 1000 Kämpfe Aria gegen einen Troll (90 Leben, Stärke 16) laufen und gib die Siegquote aus. Warum musst du für jeden Kampf neue Objekte erzeugen?
- Experiment: Welches maxLeben braucht Aria (zweiter Konstruktor, Stärke 10), damit sie mindestens 90 % der Kämpfe gewinnt?

Side-Quest: freiwillig und ohne Punkte – stellt eure Siegquoten im Kurs vor.